Adam Shostack's Four Question Framework - MyGameList

2. What Can Go Wrong?

Several security and privacy risks can affect MyGameList:

- Attackers may try to access, edit, or delete another user's game list, profile, reviews, uploaded images, or notifications.
- Weak authentication could allow account takeover through stolen passwords, brute-force login attempts, or insecure session or JWT handling.
- Users may upload malicious files disguised as images, oversized files, or files that could exploit the server or browser.
- Public reviews, notes, bios, comments, and imported game metadata could contain stored XSS payloads if user input is not sanitized.
- Moderators or administrators may abuse elevated permissions to hide content, view logs, manage users, or alter catalog data improperly.
- Admin log viewing could expose sensitive data such as passwords, tokens, API keys, session IDs, or internal server details.
- External game metadata APIs could return malicious, incorrect, or unexpected data.
- Bulk update actions could accidentally update games that do not belong to the authenticated user.
- Public profile visibility settings may fail and expose private game entries or personal information.
- Delete requests could be forged or manipulated to delete another user's game entry.
- Database errors during sequential updates could leave inconsistent user statistics, activity feeds, or game statuses.

3. What Are We Going To Do About It?

Recommended protections for MyGameList:

- Require authentication for all personal actions such as adding, editing, deleting, uploading, and bulk updating game entries.
- Enforce authorization checks on the backend for every protected action, especially ownership checks for game entries.
- Store passwords using strong password hashing, such as bcrypt or Argon2.
- Use secure session or JWT handling, including expiration, signing, HTTPS-only transport, and protection against token leakage.
- Validate all user input on both the frontend and backend.
- Sanitize or escape user-generated content before displaying reviews, bios, notes, comments, and imported metadata.
- Restrict uploads to allowed image types: `.jpg`, `.jpeg`, `.png`, and `.webp`.
- Validate uploaded files by MIME type, file extension, file size, and preferably actual file signature.
- Rename uploaded files using safe generated names instead of user-provided filenames.
- Store uploaded files outside executable server paths.
- Use role-based access control for guests, gamers, moderators, and administrators.
- Limit admin log viewing to predefined safe log categories instead of arbitrary file paths.
- Mask secrets in logs, including passwords, API keys, tokens, and session IDs.
- Keep an audit trail for sensitive admin and moderator actions.
- Validate and sanitize metadata imported from external game APIs before storing or displaying it.
- Use database transactions for bulk updates so either all related changes succeed or all are rolled back.

- Enforce visibility settings when displaying public profiles and public game lists.
- Use CSRF protection if cookie-based authentication is used.
- Rate-limit login, registration, uploads, search, and external API request features.

4. Did We Do A Good Job?

To evaluate whether the security work is good enough, MyGameList should be tested and reviewed against the risks above:

- Verify that users cannot view, edit, delete, or bulk update another user's game entries.
- Test registration, login, password change, and session expiration behavior.
- Confirm that passwords are hashed and never stored or logged in plaintext.
- Try uploading invalid files, oversized files, renamed executables, and malformed images.
- Test for stored XSS in reviews, notes, bios, comments, game titles, and imported metadata.
- Confirm that profile visibility settings correctly hide private game entries.
- Check that moderators cannot perform administrator-only actions.
- Check that administrators can only view approved log categories.
- Review logs to confirm secrets are masked.
- Test external API failures, malformed responses, and suspicious metadata.
- Test bulk updates with mixed valid and invalid game entry IDs.
- Confirm database rollback works if one step in a bulk update fails.
- Review audit logs for admin and moderator actions.
- Run automated security tests and manual abuse-case testing.
- Perform code review focused on authentication, authorization, uploads, logging, and public content display.